



Project: Software Engineering

(DLMCSPSE01)

In M.Sc. Computer Science

Submitted by

Mayank Pathak

With Matriculation Number
42308585

Under the supervision of
Prof. Dr. Holger Klus

And submitted on
Date: 12-04-2026

Table of Contents

Table of Contents	2
Abstract.....	4
1. Project Overview.....	5
1.1 Problem Statement	5
1.2 Proposed Solution	5
1.3 Expected Impact	5
1.4 Project Summary Table.....	6
2. Requirements	7
2.1 Functional Requirements	7
2.2 Non-Functional Requirements	7
2.3 Glossary	8
3. System Design and Architecture	9
3.1 Overview	9
3.2 Building Block View	9
3.3 Technical Architecture Diagram	9
3.4 Layer Descriptions	10
3.4.1 Presentation Layer	10
3.4.2 Application Logic Layer	10
3.4.3 Persistence Layer	10
3.4.4 Supporting Libraries and Tools.....	10
4. Data Model	12
.....	12
4.1 Entity Descriptions	13
5. Implementation	14
5.1 Development Methodology	14
5.2 Development Iterations	14
5.3 Technology Stack.....	14
5.4 URL Route Structure.....	15
6. Planning and Scheduling Process	16
6.1 Key Design Decisions	16
7. Features in Detail.....	18
7.1 User Authentication	18
7.2 Subject Management	18
7.3 Assignment Management	19
7.4 Calendar Import (.ics).....	19
7.5 Import Subjects from Calendar	20
7.6 Study Recommendations	20

7.7 Planner and Study Sessions	20
7.8 Weekly Calendar View	21
7.9 Application Navigation.....	21
8. Testing and Validation	23
8.1 Testing Approach	23
8.2 Test Scenarios	23
8.3 Requirements Traceability Matrix.....	24
9. Known Issues and Limitations	25
9.1 Resolved Issues.....	25
9.2 Current Limitations	25
10. Reflection and Lessons Learned	26
10.1 What Went Well	26
10.2 Challenges Encountered.....	26
10.3 Requirement Refinements During Development.....	26
10.4 What Would Be Done Differently	26
11. Future Enhancements	28
12. Project Information and Installation	29
12.1 Repository	29
12.2 Installation Instructions.....	29
13. Conclusion	30

Abstract

StudyFlow is a browser-based academic workload planning system designed to help university students manage assignments, class schedules, and study planning in one integrated platform. The system addresses a common student problem: knowing that deadlines exist but struggling to prioritize tasks, allocate study time effectively, and plan around a busy class timetable.

The application was built using Python FastAPI as the backend framework, Jinja2 for server-side rendered templates, SQLAlchemy as the ORM, and SQLite for data persistence. Key features include user authentication, subject and assignment management, .ics calendar file import, rule-based study recommendations, a planner that generates study sessions, and an interactive weekly calendar view powered by FullCalendar.

This final report consolidates all three project phases: the conception and requirements phase (P1), the development and reflection phase (P2), and the testing and deployment phase (P3). It presents the full system design, architecture, implementation details, testing strategy, and lessons learned.

1. Project Overview

1.1 Problem Statement

University students are required to manage multiple overlapping responsibilities simultaneously: attending lectures, completing assignments, preparing for assessments, and meeting submission deadlines. While institutional portals provide access to course materials and timetables, they rarely offer personalized workload planning support. Students frequently know a deadline is approaching but find it difficult to decide which task to prioritize, how much time each task requires, and how to fit study time around their existing lecture schedule.

This problem is amplified when several deadlines converge or when a student's timetable is already heavily occupied. Poor planning under these conditions leads to missed deadlines, rushed submissions, increased academic stress, and inefficient study habits.

1.2 Proposed Solution

StudyFlow is a purpose-built web application that addresses this problem by combining subject and assignment management, calendar-based timetable import, intelligent recommendation generation, and a visual weekly planning view into one cohesive tool. Rather than simply listing tasks, StudyFlow reasons about the student's schedule and workload together, helping them understand not only what needs to be done but when and how to do it given their available time.

1.3 Expected Impact

- Improved academic organization through a single platform for subjects, assignments, and schedules
- More realistic workload planning by combining due date urgency with timetable intensity
- Reduced last-minute work through proactive study session generation
- Improved time management and reduced academic stress
- More consistent study habits through visual, interactive planning

1.4 Project Summary Table

Attribute	Detail
Project Name	StudyFlow — Academic Workload Planning System
Student	Mayank Pathak (42308585)
Platform	Web browser-based application
Backend	Python, FastAPI
Frontend / Templates	Jinja2 server-rendered HTML
Database	SQLite via SQLAlchemy ORM
Calendar UI	FullCalendar (JavaScript library)
Calendar Parsing	icalendar Python library (.ics files)
Version Control	GitHub
Development Methodology	Iterative, agile-inspired, feature-driven

2. Requirements

2.1 Functional Requirements

ID	Requirement	Description
FR1	User Authentication	Users can sign up, log in, and log out securely. All features are scoped to individual user accounts.
FR2	Subject Management	Users can create, edit, delete, and view academic subjects with code, name, and color.
FR3	Assignment Management	Users can create, edit, delete, view, and filter assignments with due dates, priorities, status, and estimated study hours.
FR4	Calendar Import (.ics)	Users can upload an .ics timetable file; the system parses and stores class events in the database.
FR5	Subject Import from Calendar	Users can review and selectively import candidate subjects extracted from parsed calendar entries.
FR6	Study Recommendations	The system generates rule-based study recommendations based on due dates, assignment priorities, estimated hours, and class schedule intensity.
FR7	Planner / Study Sessions	The system generates suggested study sessions and displays them in planner and calendar views.
FR8	Weekly Calendar View	Class events and study sessions are displayed together in an interactive weekly calendar (FullCalendar).
FR9	Adjustable Study Sessions	Users can drag and resize study sessions in the calendar view; class events remain fixed.

2.2 Non-Functional Requirements

ID	Category	Requirement
NFR1	Usability	The application must be self-explanatory and operable through a standard browser without technical knowledge.
NFR2	Performance	Core operations must respond quickly for a student-sized dataset.
NFR3	Maintainability	Features must be modular and independently improvable.
NFR4	Reliability	Uploaded calendar data and generated study sessions must be stored consistently.
NFR5	Traceability	Source code must be maintained in GitHub with meaningful commit history.

2.3 Glossary

Term	Definition
Subject	An academic module or course such as Algorithmics or Computer Science and Society.
Assignment	A task linked to a subject, carrying a due date, priority, status, and estimated study effort.
Class Event	A lecture or timetable entry imported from an .ics calendar file.
Study Session	A suggested or manually adjusted study block generated by the planner.
Recommendation	A rule-based planning suggestion derived from workload urgency and schedule intensity.
Planner	The module that transforms assignments and schedule constraints into daily study allocations.
.ics File	A standard iCalendar format file used to import timetable data from external sources.

3. System Design and Architecture

3.1 Overview

StudyFlow follows a three-layer architecture comprising a Presentation Layer, an Application Logic Layer, and a Persistence Layer. This separation of concerns ensures each aspect of the system can be developed, tested, and maintained independently. A fourth category of Supporting Libraries and Tools underpins both the application and persistence layers.

3.2 Building Block View

Figure 1 shows the high-level building block structure of StudyFlow, illustrating how the student user interacts with the system through each layer.

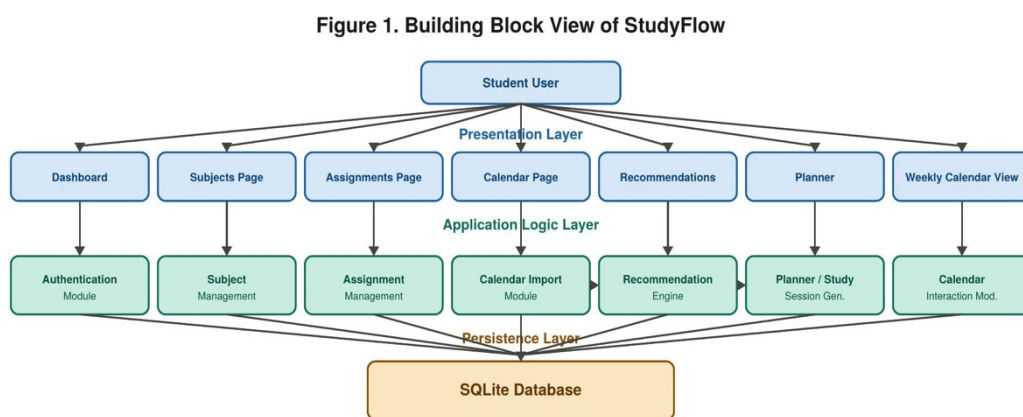


Figure 1. Building Block View of StudyFlow

3.3 Technical Architecture Diagram

Figure 2 shows the full technical architecture of StudyFlow, including how the browser-based client layer interacts with the FastAPI application, how business logic is separated into route and logic layers, and how persistent data flows through SQLAlchemy into the SQLite database.

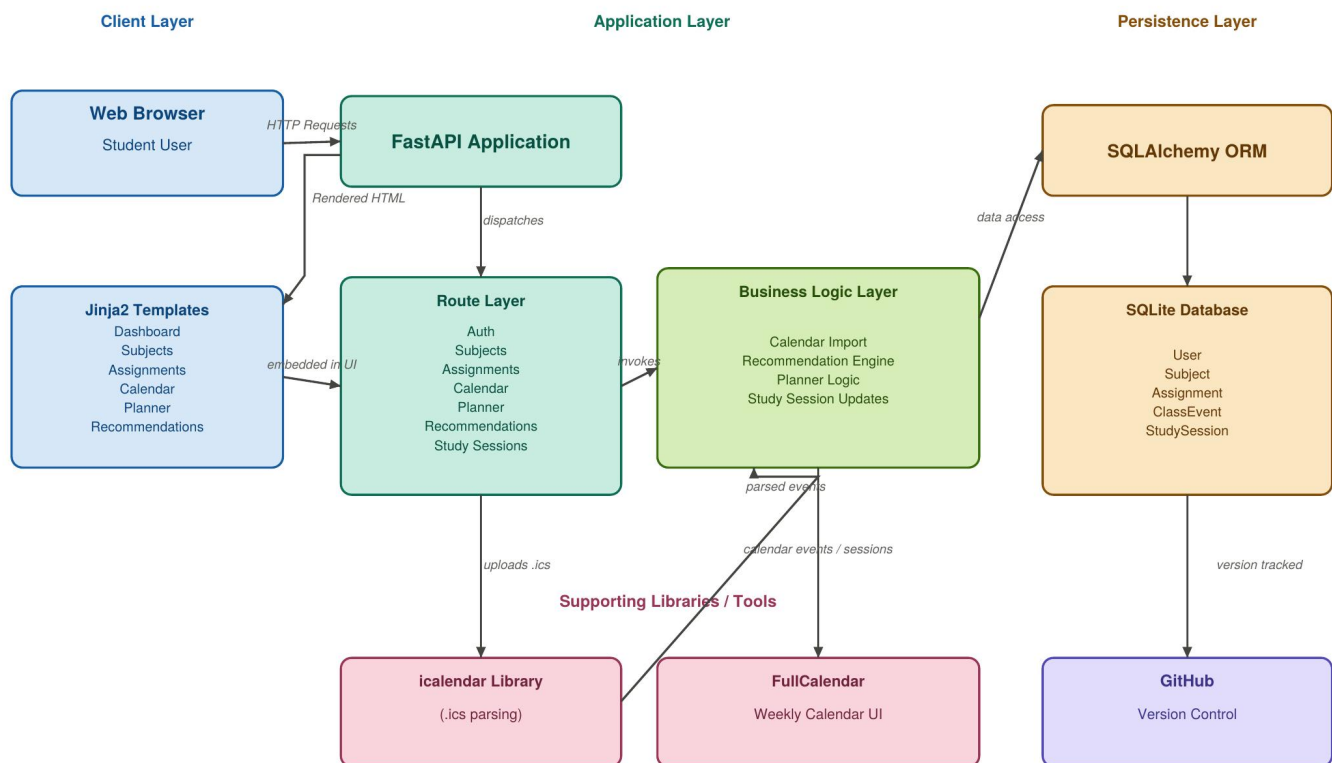


Figure 2. Technical Architecture Diagram of StudyFlow

3.4 Layer Descriptions

3.4.1 Presentation Layer

The presentation layer consists of browser-rendered Jinja2 HTML templates. Each feature in the system has a corresponding view: Dashboard, Subjects, Assignments, Calendar, Import Subjects, Recommendations, Planner, and Weekly Calendar View.

3.4.2 Application Logic Layer

The application layer is divided into a Route Layer (FastAPI route handlers) and a Business Logic Layer comprising four modules: Calendar Import (parses .ics and extracts subjects), Recommendation Engine (evaluates urgency and schedule intensity), Planner Logic (generates study sessions from assignment data and schedule gaps), and Study Session Updates (handles drag-and-resize from FullCalendar).

3.4.3 Persistence Layer

Data is stored in a SQLite database accessed through SQLAlchemy ORM models. Core entities are User, Subject, Assignment, ClassEvent, and StudySession.

3.4.4 Supporting Libraries and Tools

- icalendar — parses uploaded .ics timetable files into structured class event records

- FullCalendar — provides the interactive weekly calendar UI with drag-and-drop and resize support
- GitHub — version control and source code hosting throughout all project phases

4. Data Model

Figure 3 shows the core entity-relationship model of StudyFlow. Every data entity is linked to a User, ensuring complete data isolation between different student accounts.

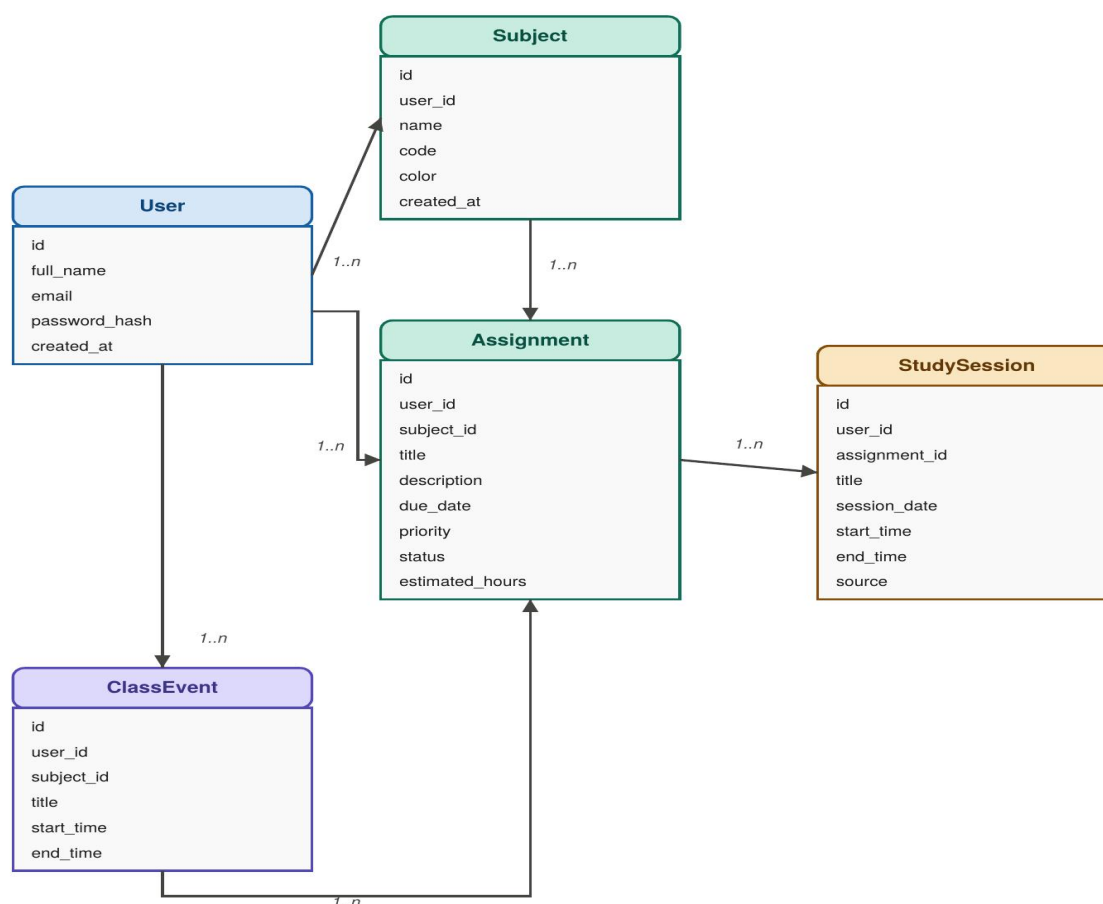


Figure 3. Core Data Model of StudyFlow

4.1 Entity Descriptions

Entity	Key Fields and Purpose
User	id, full_name, email, password_hash, created_at. Central entity; all other records belong to a user.
Subject	id, user_id, name, code, color, created_at. Represents an academic module. Linked to assignments and class events.
Assignment	id, user_id, subject_id, title, description, due_date, priority, status, estimated_hours. Core planning unit; drives recommendations and study sessions.
ClassEvent	id, user_id, subject_id, title, start_time, end_time. Created from .ics import; represents scheduled lectures. Fixed in the calendar view.
StudySession	id, user_id, assignment_id, title, session_date, start_time, end_time, source. Generated by the planner; user-adjustable in FullCalendar.

5. Implementation

5.1 Development Methodology

The project followed an iterative, agile-inspired development methodology. Rather than implementing all features at once, the system was built in feature-driven iterations with each producing a usable intermediate version. This approach reduced implementation risk, allowed for interface improvements between iterations, and made it possible to incorporate new ideas without redesigning from scratch.

5.2 Development Iterations

Iteration	Focus	Key Deliverables
Iteration 1	Authentication, Subjects, Assignments, Dashboard	User login/signup, subject CRUD, assignment CRUD, dashboard summaries
Iteration 2	Calendar Upload and Subject Import	ICS file parsing, class event storage, subject candidate extraction and review UI
Iteration 3	Recommendations, Planner, Weekly Calendar	Rule-based recommendations, study session generation, FullCalendar weekly view
Iteration 4	UI Polish, Interactions, Documentation	Drag-and-resize study sessions, UI improvements, documentation, deployment prep

5.3 Technology Stack

Technology	Reason for Selection
FastAPI (Python)	Lightweight and modular Python web framework suited to structured route handling and rapid development.
Jinja2 Templates	Server-rendered approach that keeps the frontend simple without the overhead of a full frontend framework.
SQLAlchemy ORM	Structured ORM support for managing entities with clean model definitions and relationship handling.
SQLite	Simple, file-based database appropriate for development and demonstration without external server dependencies.
icalendar (Python)	Standard library for parsing .ics timetable files and extracting structured class event records.
FullCalendar (JS)	Provides a proper weekly calendar UI with drag-and-drop and resize interactions, replacing an earlier list-based display.
GitHub	Version control, source code hosting, and traceability of all project phases.

5.4 URL Route Structure

Route	Method	Description
/auth/signup	GET/POST	User registration
/auth/login	GET/POST	User login
/auth/logout	GET	User logout
/dashboard	GET	Assignment summary dashboard
/subjects	GET	List all subjects
/subjects/new	GET/POST	Create a new subject
/subjects/{id}/edit	GET/POST	Edit an existing subject
/subjects/{id}/delete	POST	Delete a subject
/assignments	GET	List and filter assignments
/assignments/new	GET/POST	Create a new assignment
/assignments/{id}/edit	GET/POST	Edit an assignment
/calendar	GET/POST	Upload .ics file
/calendar/import-subjects	GET/POST	Review and import subjects from calendar
/recommendations	GET	View study recommendations
/planner	GET	View generated study sessions
/calendar-view	GET	Interactive FullCalendar weekly view
/study-sessions/{id}	PUT	Update study session time (drag/resize)

6. Planning and Scheduling Process

Figure 4 illustrates the end-to-end planning workflow in StudyFlow, from the initial .ics file upload through to the final interactive weekly calendar view. The process is user-controlled at each major decision point while automating computationally intensive reasoning steps.

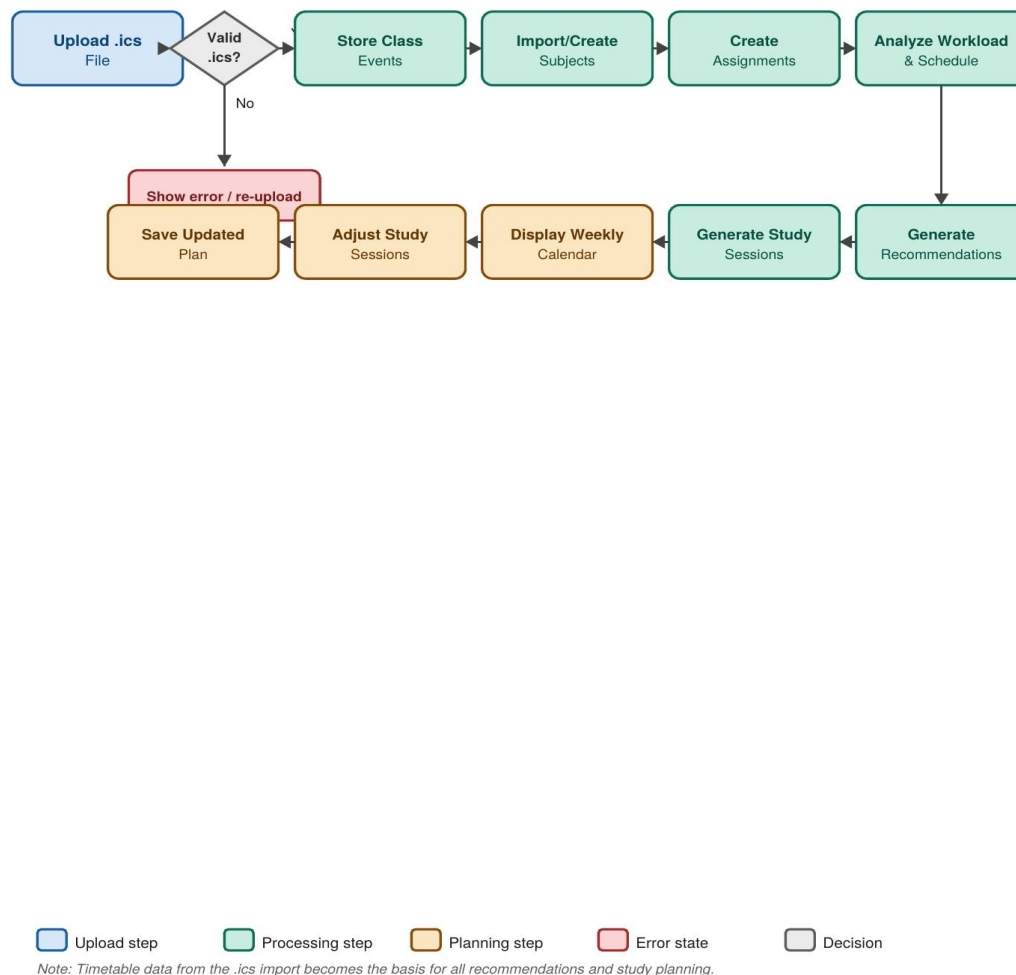


Figure 4. StudyFlow Planning and Scheduling Process Flow

6.1 Key Design Decisions

- Calendar-based subject import is user-controlled (not fully automatic) because timetable titles vary in format and quality across institutions and semesters.
- Recommendation and planner logic is rule-based rather than an optimization engine, ensuring transparency and explainability appropriate for a student planning tool.
- Class events are fixed in the calendar while study sessions are user-adjustable, reflecting the real distinction between mandatory lectures and flexible study time.

- FullCalendar replaced an earlier list-based schedule display, significantly improving usability and making time-blocking intuitive and visual.

7. Features in Detail

7.1 User Authentication

The authentication module was implemented first so all subsequent features could be scoped to individual users. The system supports secure signup, login, and logout. Passwords are stored as hashes and sessions are managed server-side.

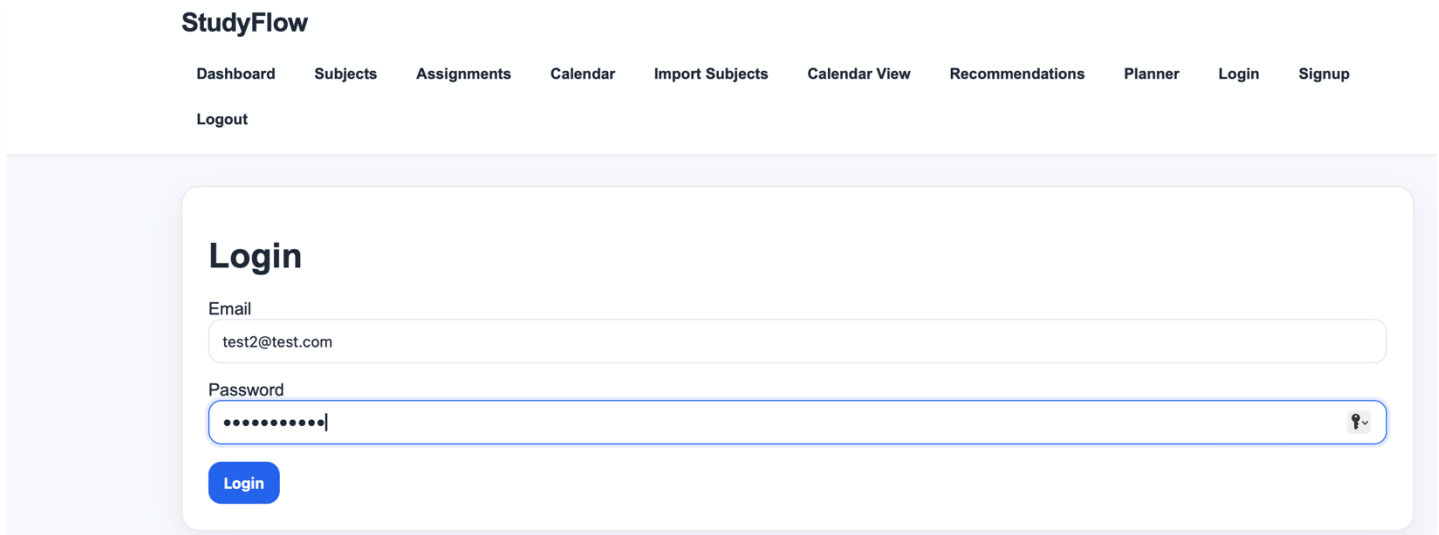


Figure 5. StudyFlow Login Page

7.2 Subject Management

Students manage academic subjects through a CRUD interface. Each subject has a Subject Code (e.g. CSEMCSA01), a Subject Name (e.g. Algorithmics), and a color used to visually distinguish it in the calendar. Subjects can be created manually or imported from a calendar file.

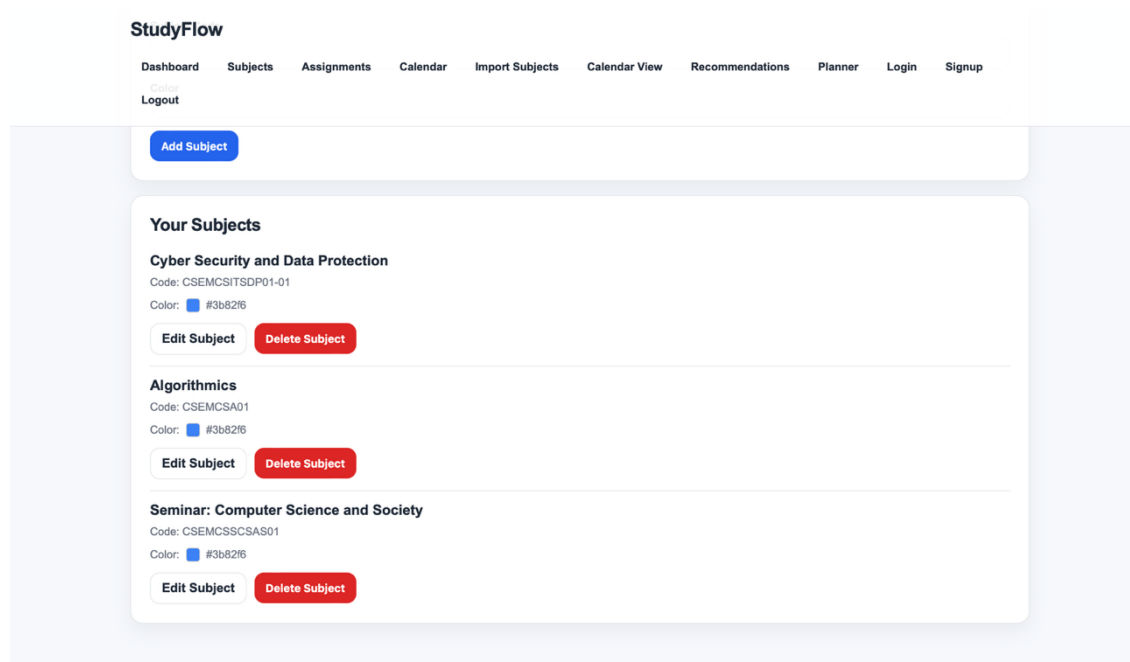


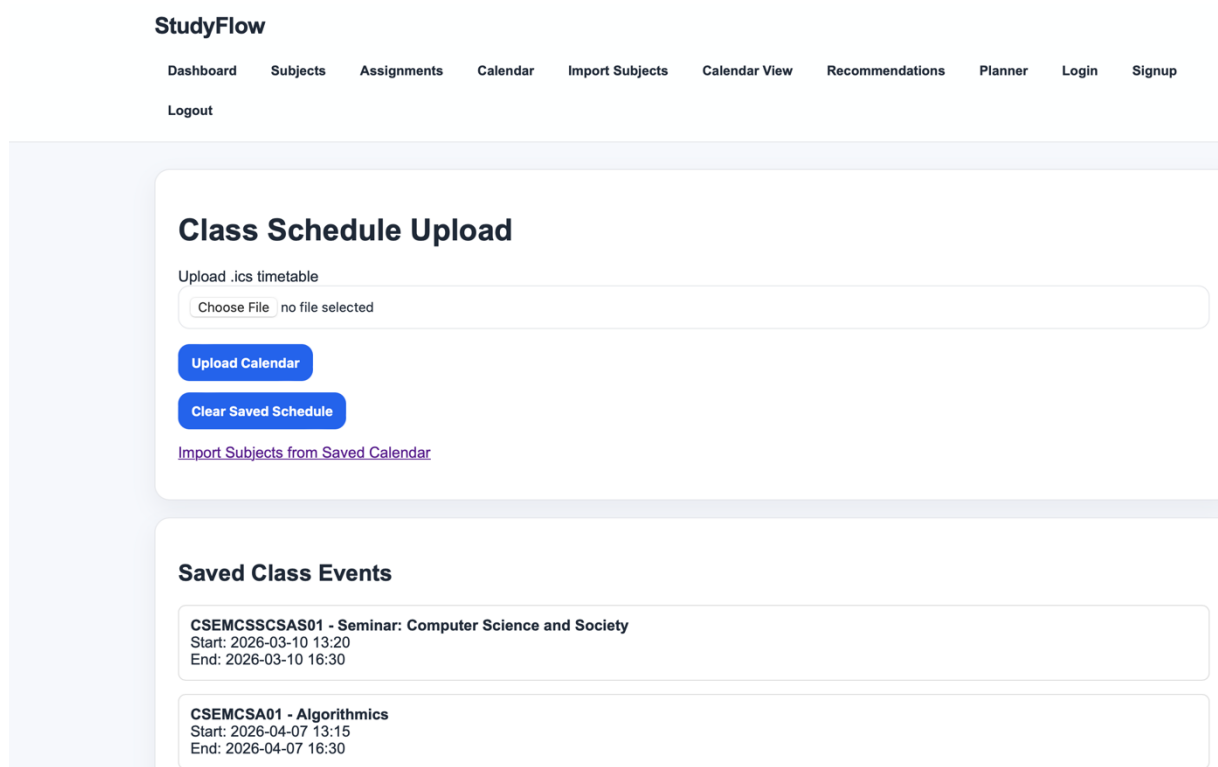
Figure 6. StudyFlow Subject Management Page

7.3 Assignment Management

Assignments are the core planning unit of StudyFlow. Each assignment is linked to a subject and carries a title, description, due date, priority (low / medium / high), status (pending / in progress / completed), and an estimate of study hours required. The dashboard provides a quick overview of overdue and due-soon items. Assignments can be filtered by status, priority, and subject.

7.4 Calendar Import (.ics)

Students upload their institutional timetable as an .ics file. The icalendar Python library parses the file and extracts individual class events (title, start time, end time). These events are stored as ClassEvent records and are immediately visible in the weekly calendar view as fixed blue blocks.



StudyFlow

Dashboard Subjects Assignments Calendar Import Subjects Calendar View Recommendations Planner Login Signup

Logout

Class Schedule Upload

Upload .ics timetable

Choose File no file selected

Upload Calendar

Clear Saved Schedule

[Import Subjects from Saved Calendar](#)

Saved Class Events

CSEMSSCSAS01 - Seminar: Computer Science and Society
Start: 2026-03-10 13:20
End: 2026-03-10 16:30

CSEMCSA01 - Algorithmics
Start: 2026-04-07 13:15
End: 2026-04-07 16:30

Figure 7. StudyFlow Calendar upload

7.5 Import Subjects from Calendar

After uploading a timetable, the system analyses the class event titles and generates candidate subjects. The Import Subjects page presents each detected calendar entry with its original raw title for reference, alongside editable Subject Code and Subject Name fields pre-populated with suggested values. The student selects which entries to import using checkboxes and submits. Only selected entries are saved as subjects.

7.6 Study Recommendations

The Recommendations module applies rule-based heuristics to the student's current data: identifying the most urgent pending assignment, calculating total pending study hours, identifying lighter class days as optimal study days, flagging assignments due within three days as urgent, and suggesting specific tasks for specific days based on urgency and schedule availability.

7.7 Planner and Study Sessions

The Planner module transforms assignment data and schedule constraints into concrete suggested study sessions. It distributes study effort across available time slots, considering class days and estimated hours per assignment. Generated sessions appear as green blocks in the weekly calendar view.

7.8 Weekly Calendar View

The Weekly Calendar View provides an interactive visual representation of the student's full schedule. Blue blocks represent class events imported from the .ics file and are fixed. Green blocks represent generated study sessions and are user-adjustable via drag and resize. All changes persisted via a REST endpoint.

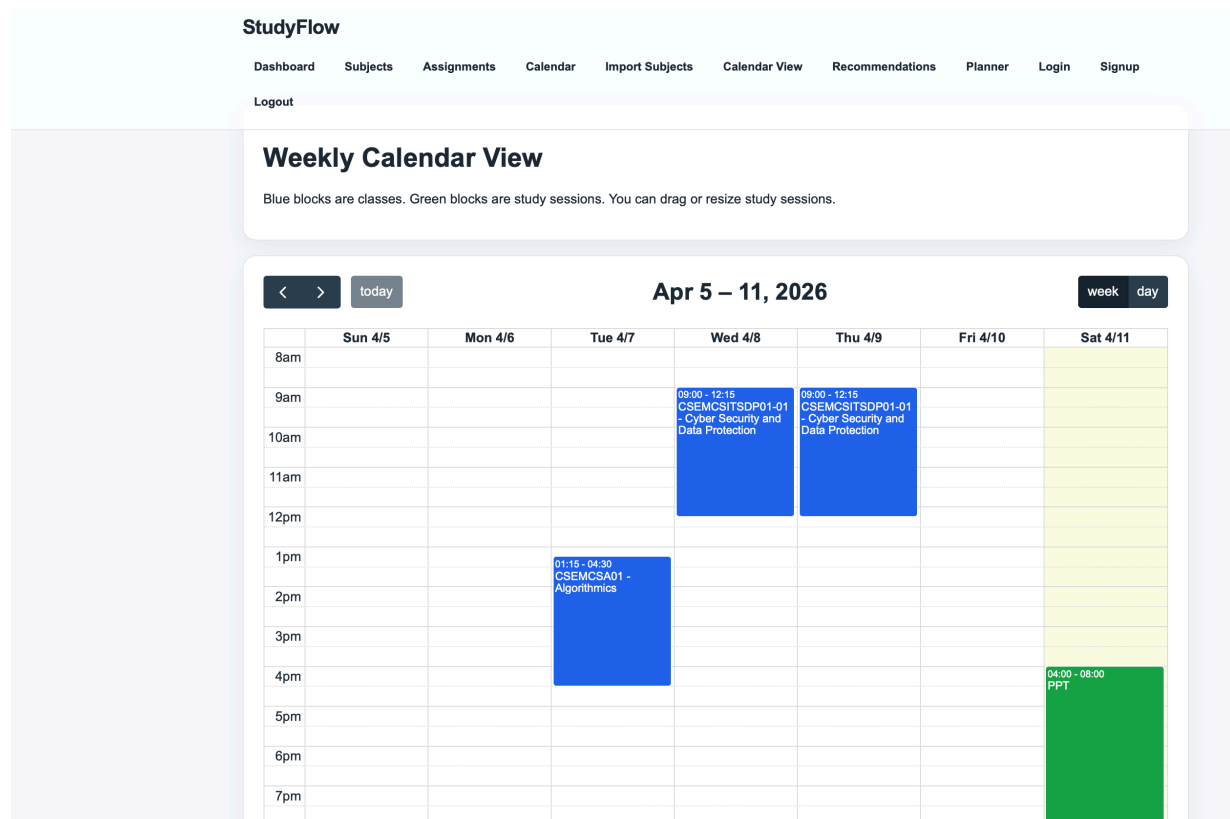


Figure 8. StudyFlow Calendar view

7.9 Application Navigation

The application navigation bar (visible in Figures 5 and 6) provides direct access to all major features: Dashboard, Subjects, Assignments, Calendar, Import Subjects, Calendar View, Recommendations, and Planner. Login and Logout links are also provided in the navbar, enabling fast switching between authenticated states.

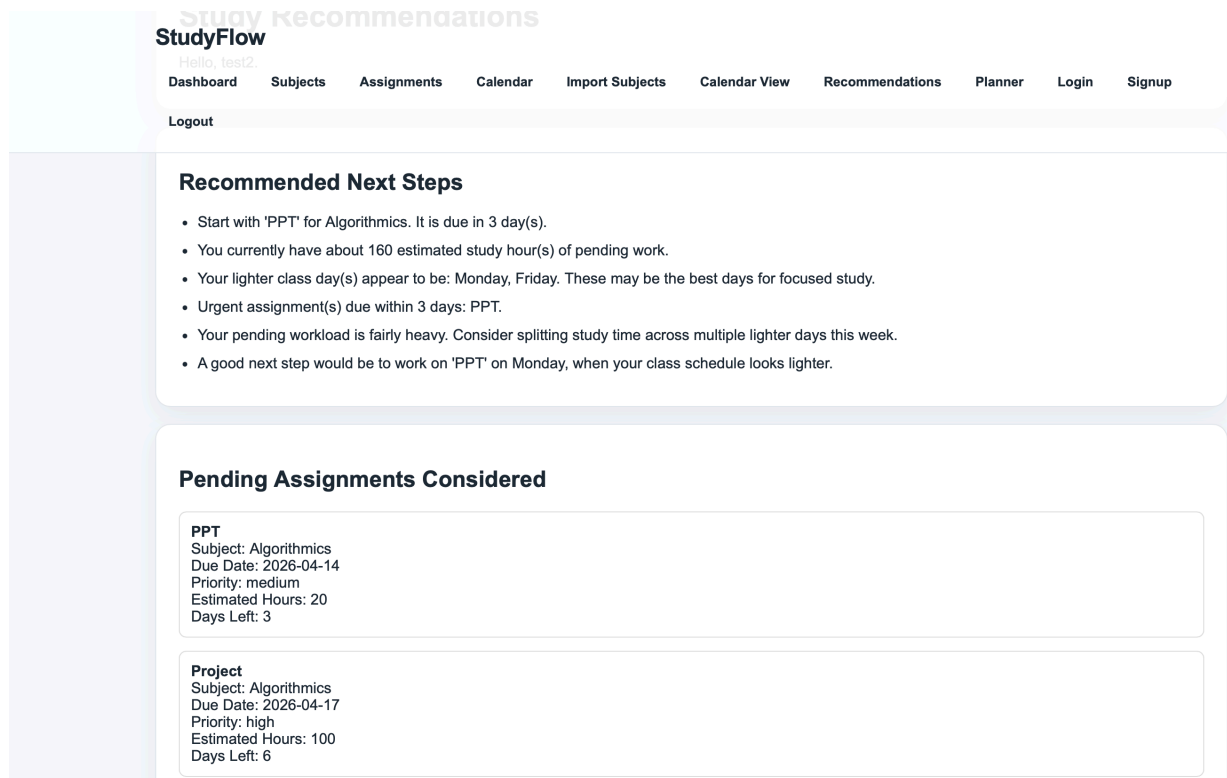


Figure 9. StudyFlow Recommendation feature

8. Testing and Validation

8.1 Testing Approach

Testing was conducted across multiple levels aligned with the functional and non-functional requirements. The overall approach prioritized correctness of feature behavior, verification of data integrity, and validation of usability through manual end-to-end testing across all implemented features.

8.2 Test Scenarios

Scenario	Requirement	Expected Outcome
User signup and login	FR1	Account created; login redirects to dashboard
Login with invalid credentials	FR1	Error message shown; no access granted
Create, edit, and delete a subject	FR2	Subject list updated correctly
Create assignment with due date	FR3	Assignment appears in list and dashboard
Filter assignments by status	FR3	Correct subset displayed
Upload a valid .ics timetable file	FR4	Class events stored and visible in calendar
Upload invalid .ics file	FR4	Error shown; no partial data saved
Import subjects from calendar entries	FR5	Selected subjects created correctly
View study recommendations	FR6	Recommendations reflect actual urgency
Generate and view study sessions	FR7	Sessions created and visible in planner
Drag/resize a study session	FR9	Updated time persisted; class events unchanged
Multiple users — data isolation	NFR1	User A cannot see User B's data
Dashboard reflects completed assignments	FR3/NFR2	Status updates correctly

8.3 Requirements Traceability Matrix

ID	Requirement	Test Method	Status
FR1	User authentication (signup, login, logout)	Manual end-to-end	PASS
FR2	Subject CRUD	Manual CRUD testing	PASS
FR3	Assignment CRUD and filtering	Manual CRUD + filter testing	PASS
FR4	ICS file upload and class event storage	Valid and invalid file tests	PASS
FR5	Subject import from calendar	Import review UI test	PASS
FR6	Study recommendations	Verified against known data	PASS
FR7	Study session generation	Planner output verified	PASS
FR8	Weekly calendar display	Visual browser inspection	PASS
FR9	Drag and resize study sessions	Interactive browser testing	PASS
NFR1	Usability — browser-based	User walkthrough testing	PASS
NFR2	Performance — responsive	Manual timing of key operations	PASS
NFR3	Maintainability — modular codebase	Code review audit	PASS
NFR4	Reliability — data persists correctly	Database inspection	PASS
NFR5	Traceability — GitHub history	Repository inspection	PASS

9. Known Issues and Limitations

9.1 Resolved Issues

Issue	Resolution
Calendar entry text overflowing outside the subject import card	Simplified to inline layout: checkbox and entry text on one line without flex container
Subject Code input field overflowing card width	Applied width: 100% and box-sizing: border-box to all input fields
Vertical text rendering caused by flex layout on the entry row	Removed flex and nested label wrappers; used plain inline elements
Original calendar entry text invisible after layout changes	Entry displayed inline after checkbox label with no wrapper divs
List-based calendar display not communicating time blocking	Replaced with FullCalendar interactive weekly view

9.2 Current Limitations

- SQLite is appropriate for development and demonstration but should be replaced with PostgreSQL for production deployment.
- The application does not support high concurrency due to SQLite's write-lock behavior.
- Recommendation logic is rule-based and does not adapt to individual student behavior over time.
- The login email field uses browser-level email format enforcement while the system accepts any string — this validation inconsistency should be corrected.
- No mobile-optimized layout exists; the application is designed for desktop browser use.
- Session security uses basic server-side session cookies; production deployment should use token-based authentication.

10. Reflection and Lessons Learned

10.1 What Went Well

- The iterative development methodology was highly effective. Delivering a working system at the end of each iteration made it easy to test features incrementally and avoid accumulating technical debt.
- FastAPI's clean route structure made it straightforward to keep business logic separated from HTTP handling.
- Replacing the list-based schedule display with FullCalendar was one of the most impactful single changes made during the project. The interactive calendar made the planning functionality tangible and usable in a way that a plain list could not achieve.
- SQLAlchemy's ORM approach made the data model easy to evolve across iterations without manual SQL migrations.
- Keeping recommendation logic rule-based rather than building a complex optimization engine was correct for the project scope — recommendations are transparent and understandable.

10.2 Challenges Encountered

- Parsing .ics files from real institutional calendar exports required handling inconsistent title formats, extra metadata, and time zone variations not apparent from the specification.
- The subject import UI required multiple iterations to get right. Early versions had layout issues where text overflowed the card boundary, and flex layout caused vertical text rendering. The final solution was the simplest possible: plain inline HTML elements without any flexbox.
- Integrating FullCalendar with server-rendered Jinja2 templates required careful design of the data pipeline from the FastAPI backend to the JavaScript frontend, including dedicated JSON endpoints alongside the standard HTML routes.
- Planner logic had to balance distributing study time evenly against prioritizing urgent assignments, requiring several refinements to the scheduling algorithm.

10.3 Requirement Refinements During Development

Several requirements became more specific during implementation than the conception phase had anticipated. Assignment management expanded from basic CRUD to include filtering, dashboard summaries, and subject-linked organization. Calendar integration evolved beyond simple file import: uploaded timetable data became the foundation for subject extraction, recommendation logic, planner generation, and calendar visualization. Planning-related requirements were refined the most — the system evolved from displaying tasks and deadlines to generating recommendations, creating suggested study sessions, and displaying those sessions alongside lectures in an interactive calendar.

10.4 What Would Be Done Differently

- Introduce proper email validation from the start rather than leaving the login form with inconsistent validation.

- Design the frontend with mobile responsiveness from the beginning using a CSS framework.
- Use PostgreSQL from the start rather than SQLite, to avoid the transition effort required for production.
- Write automated unit tests for the recommendation and planner logic from the beginning of Iteration 3.

11. Future Enhancements

Enhancement	Description
PostgreSQL migration	Replace SQLite with PostgreSQL for production-grade persistence and concurrent access.
Email notifications	Send email or push notifications when assignment deadlines approach.
Direct .ics / Google Calendar sync	Live synchronization with Google Calendar or Outlook rather than manual file upload.
Mobile-responsive design	Enrich all layouts for smartphone and tablet screens.
AI-powered recommendations	Replace rule-based recommendations with a model that adapts to individual study patterns.
Schedule conflict detection	Automatically detect and flag timetable clashes on import.
Export functionality	Allow students to export their timetable and study plan as PDF or image.
Collaborative features	Group study session planning and shared assignment tracking.
Dark mode	Optional dark UI theme for comfort during extended study sessions.
Automated test suite	Full unit and integration test coverage for recommendation and planner logic.

12. Project Information and Installation

12.1 Repository

The full source code for StudyFlow is available at:

<https://github.com/mayankpathak-iu/software-engineering-project>

The repository is organized into an `app/` directory (FastAPI application, Jinja2 templates, SQLAlchemy models, route handlers) and a `docs/` directory (project documentation and diagrams).

12.2 Installation Instructions

1. Clone the repository: `git clone https://github.com/mayankpathak-iu/software-engineering-project`
2. Install Python dependencies: `pip install -r requirements.txt`
3. Run the application: `uvicorn app.main:app --reload`
4. Open a browser and navigate to: `http://localhost:8000`
5. Sign up for an account and begin by uploading a `.ics` timetable file from your institution.

13. Conclusion

StudyFlow successfully delivers a browser-based academic workload planning system that addresses a genuine problem faced by university students. By combining subject and assignment management with calendar-based timetable integration, rule-based study recommendations, and an interactive weekly calendar view, the application gives students a practical tool for managing their academic workload in relation to their actual schedule.

The system was developed across three phases following an iterative, agile-inspired methodology. Each iteration produced a working, testable version of the application, enabling progressive refinement of both functionality and user interface design. All nine functional requirements and five non-functional requirements defined in the conception phase have been implemented and validated.

The key technical achievements include the FullCalendar integration for interactive scheduling, the .ics parsing pipeline for calendar import, the rule-based recommendation engine, and the planner logic that translates abstract workload data into concrete, adjustable study sessions. The iterative approach also allowed for the identification and correction of significant UI bugs in the calendar import screen, where multiple layout approaches were tried before a simple inline HTML solution resolved all overflow and rendering issues.

StudyFlow demonstrates that a focused, well-scoped web application built with careful attention to user needs and iterative testing can deliver meaningful value within a course project context. The project has been a valuable learning experience in full-stack web development, software engineering methodology, and the practical challenges of turning a planning concept into a working, testable system.